

guard-track-branch-label

1 1 4 1 8 2

guard-track-branch-label.sh – § . . track+branch+alias label guard hook

Revision: 1 **Last modified:** 2026-07-11T00:00:00Z **Authority:** constitution
submodule §11.4.182 (Track+branch work-stream identity label) · §11.4.177
(inherited-by-reference) · §11.4.6 (no-guessing / honest boundary) **Classification:**
universal (§11.4.17)

Overview

scripts/hooks/guard-track-branch-label.sh is a Claude Code **PreToolUse guard hook** that enforces §11.4.182: every agent-dispatching tool call (Agent / Task / TaskCreate) MUST carry a work-stream identity label of the form (T<N>/<branch> - <alias>) at the START of its description (or, as a fallback, its subagent field), so parallel-track work (/mnt/track1..4 , each on its own branch, each driven by a distinct claude-toolkit alias) is never ambiguous.

The hook is inherited **by reference** (§11.4.177) — it lives in the constitution submodule and is wired into a consuming project's .claude/settings.json PreToolUse hooks. It is NEVER copied per project.

What it enforces

1. **FORMAT** — the label matches `^\(T[0-9]+/[^\)]+ - [^\)]+\)` (numeric track, a branch, - , an alias,) , then a space). A missing/malformed label on an agent dispatch is BLOCKED (exit 2).

2. **ALIAS-CORRECTNESS** (the §11.4.182 alias field) — a format-valid label is NOT sufficient. The label's `<alias>` MUST match the **LIVE** alias derived from `CLAUDE_CONFIG_DIR` (basename `.claude-<alias>` → `<alias>`). A hand-typed / remembered / **stale** alias (e.g. `claude4` while the live session is `claude3`) is BLOCKED (exit 2) with the CORRECT label printed.

The live alias is derived by **calling the reference labeler** `scripts/multitrack/track_branch_label.sh` — the single source of truth for the T/branch/alias derivation. The hook does not duplicate the derivation regex (DRY); the same labeler output drives both the alias check and the block message's "correct label" line.

Honest boundary (\$. .)

A `?` on **either** side is ACCEPTED, never fabricated into a verdict:

- **live alias unknown** (`CLAUDE_CONFIG_DIR` unset / non- `.claude-*`) → the hook cannot verify the alias, so any label is accepted on the alias axis;
- **caller honestly declares** `?` ((T1/main - `?`)) → accepted even when the live alias is known.

The hook BLOCKS only a **concrete** alias that **provably disagrees** with a **known** live alias. `?` is the honest value, never a bluff.

Prerequisites

- `bash` (the hook and labeler are `#!/usr/bin/env bash`).
- `git` (optional — the labeler falls back to `?` for the branch if absent).
- The sibling labeler present at `scripts/multitrack/track_branch_label.sh` (a defensive inline fallback keeps the hook functional if it is unreachable).
- `CLAUDE_CONFIG_DIR` exported by the driving session (its basename encodes the alias). Absent ⇒ live alias `?` ⇒ alias axis not enforced (honest boundary).

Usage

Wired as a PreToolUse hook (matcher `Agent | Task | TaskCreate`) in `.claude/settings.json` :

```
{ "type": "command",  
  "command": "bash \"$CLAUDE_PROJECT_DIR/constitution/scripts/hooks/guard-track-branch-label.sh\" }
```

The hook receives the tool invocation as JSON on stdin and:

- **exit 0** → allow (Claude proceeds);
- **exit 2** → BLOCK; stderr is fed back to Claude as the refusal reason and includes the CORRECT label for this checkout+session so the caller can fix it;
- any other exit → non-blocking error (never used; the defensive fallback keeps the hook to 0/2).

Derive the exact label manually with:

```
bash constitution/scripts/multitrack/track_branch_label.sh # e.g. (T1/main - claude3)
```

Edge cases

- **Non-agent tools** (`Bash` , `Read` , `Edit` , unknown tools) pass through untouched (exit 0) — the hook never breaks non-agent tools.
- **Empty / missing alias** (`(T1/main)` , `(T1/main - )`) → BLOCKED on FORMAT.
- **Branch names with -** (e.g. `feature/mistiq-vader`) — a git ref cannot contain a space, so the `-` alias separator is unambiguous.
- **Labeler unreachable** — a defensive inline derivation mirrors the labeler so the hook still emits an actionable message and never exits non-0/2.

Internal behaviour

1. Read stdin JSON; extract `tool_name` (jq if present, else an awk fallback).

2. Non-agent tool $\Rightarrow$ exit 0.
3. Extract the label from `description` (fallback `subagent`).
4. Call the labeler for the canonical `(T<N>/<branch> - <alias>)` label; extract `_live_alias` from its alias field.
5. Extract `_dispatch_alias` from the caller's label.
6. Decide: FORMAT fails $\Rightarrow$ BLOCK (format); FORMAT ok AND both aliases concrete AND they differ $\Rightarrow$ BLOCK (alias); otherwise $\Rightarrow$ exit 0.
7. On BLOCK, print the CORRECT label + how to fix; exit 2.

Tests / gates

- **Hermetic test:** `scripts/hooks/test_guard_track_branch_label.sh` (30 cases; controls `CLAUDE_CONFIG_DIR` per case for determinism — §11.4.50/§11.4.98).
- **Pre-build gate:** `scripts/gates/cm_track_branch_label.sh` (`CM-TRACK-BRANCH-LABEL` — presence/exec/parseable + doc + behavioral alias-validation).
- **Paired §1.1 mutation:** `scripts/gates/cm_track_branch_label_mutation_test.sh` (a format-only hook that skips the alias check $\rightarrow$ the gate FAILs).

Related scripts

- `scripts/multitrack/track_branch_label.sh` — the reference labeler (single source of truth for the derivation).
- `scripts/hooks/guard-work-track-binding.sh` — the §11.4.191 work $\rightarrow$ track binding guard (sibling PreToolUse hook).
- `scripts/hooks/guard-forbidden-commands.sh` — the §11.4.109 anti-forgetting guard.

Last verified: 2026-07-11 (hermetic suite 30/30 PASS; gate + §1.1 mutation clean/mutated/restored GREEN on this checkout).